

a little book

A Practical Introduction to RAG

Build Your First AI Assistant



by Richel Ohenewaa Attafuah

Richel makes tech easy

Meet Nelly



Nelly has a problem.

It's late, her deadline is in three days, and she's looking for one specific thing she knows she read a few weeks ago. A single fact. A definition, a result, one useful paragraph.

The problem is she doesn't remember where.

Was it in one of the forty PDFs on her desktop? The lecture notes? That article she saved and swore she'd read properly later? Somewhere in this pile is exactly the answer she needs, and she can feel it just out of reach.

Meet Nelly

So Nelly does what we all do.

She opens a document and hits **Ctrl+F**, but that only searches the one file she happens to have open, and she has hundreds. She'd have to already know where the answer is to find where the answer is.



So she tries pasting her question into ChatGPT. It answers instantly and confidently, and makes something up, a fact that sounds right but isn't in any of her notes. Somehow that's worse than no answer at all.



What Nelly actually needs is something that can read *all* of her documents, find the exact piece that answers her question, and reply using only what's truly in there.

She doesn't know it yet, but what Nelly needs is called **RAG**.

And by the end of this little book, you're going to build it with her, for her messy pile of notes, and for yours too.



Chapter 1: What RAG Actually Is

Let's start with the scary-looking part and make it un-scary.

RAG stands for **Retrieval-Augmented Generation**. Three intimidating words that actually describe something very simple. Let's take them one at a time, backwards, because that's the order that makes sense.

Generation just means an AI writing an answer. It's what ChatGPT does when it replies to you.

Retrieval means finding the right information first. Like Nelly digging through her notes for the one paragraph that matters, except fast, and across everything at once.

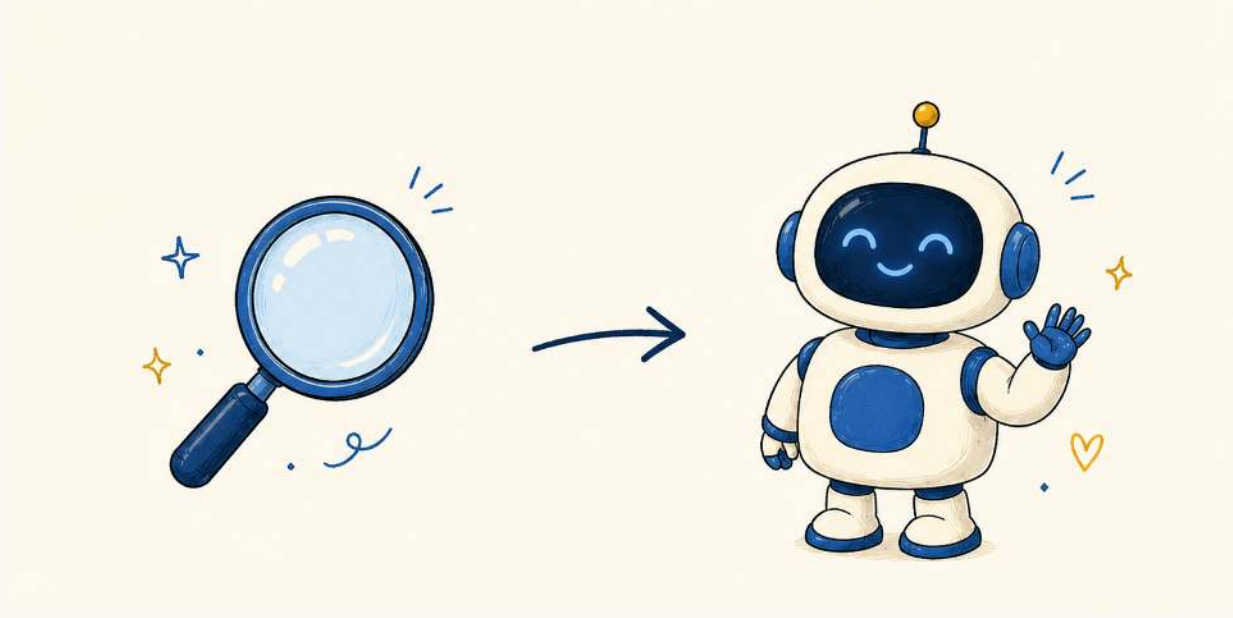
Augmented is the glue. It just means we boost the AI's answer by handing it the retrieved information before it replies.

The whole big idea

So put it back together and read it plainly:

*Find the right information first.
Then let the AI answer using it.*

That's RAG. That's the whole big idea. Everything else in this book is just *how* we do those two things well.



Remember when Nelly asked ChatGPT directly and it confidently made something up? That happens because the AI was answering from its general memory, not from her documents. It had never seen her notes. So it guessed.

RAG fixes exactly that. Instead of letting the AI guess, we show it the relevant pages first, then ask it to answer using only those. The answer comes from real, retrieved truth.

Two students, one exam

Think of it like the difference between two students taking the same exam.



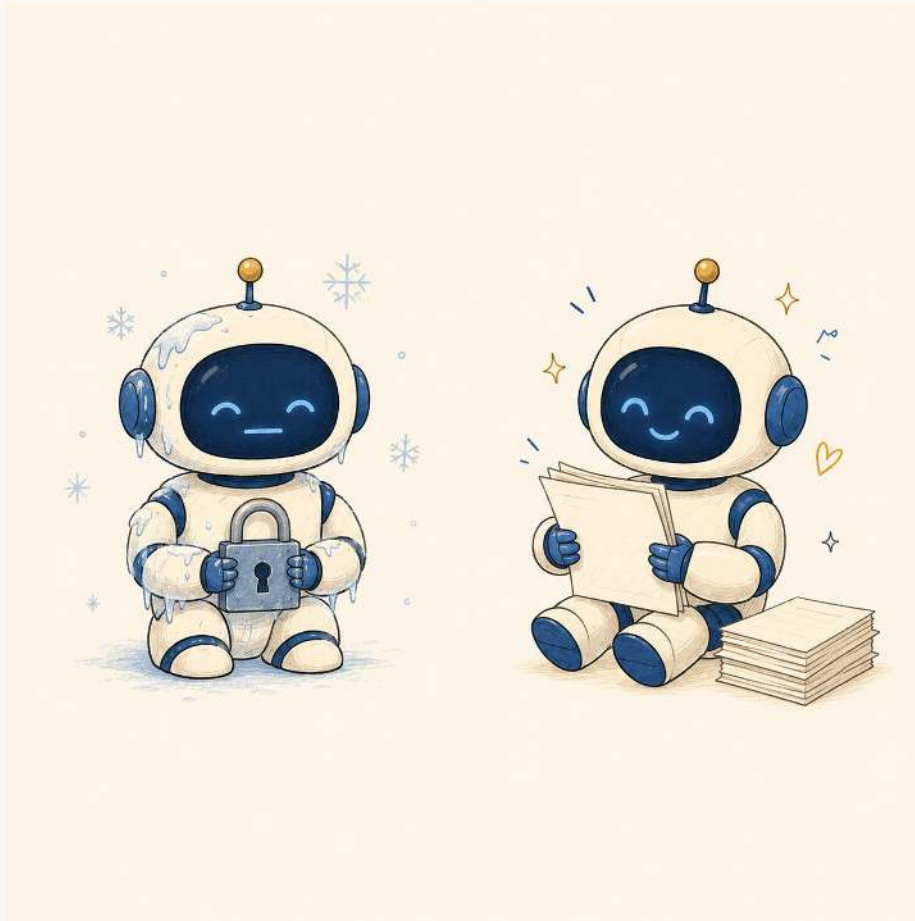
The first answers from memory alone. Sometimes right, sometimes confidently wrong, just like the AI guessing.

The second gets to open the textbook, find the exact page, and answer from what's actually written there. Same student, same brain, but now the answers are grounded in something real.

RAG turns the AI from the first student into the second one.

That's the concept the whole rest of this book is built on. Everything from here is simply learning how to give our AI the right page, at the right moment, every time.





Chapter 2: Why RAG Matters

You might be thinking: AI already answers questions. Why add all this?

Three reasons, and once you see them you can't unsee them.

It knows *your* world. A plain AI only knows what it learned during training. It has never seen Nelly's notes, your company's documents, or last week's meeting. RAG lets it answer from *your* information, not just the public internet it was trained on.

Why RAG Matters

It stays current. A trained model is frozen at the moment it was made. RAG reads from documents you can update any time, so the answer reflects today's truth, not last year's.



It tells the truth, and shows its work. Because the answer comes from real retrieved notes, you can check it against the source. No more confident nonsense. You can point at the exact note the answer came from.



That last one is the big deal. An answer you can *trust and trace* is worth far more than a clever one you have to second-guess. That's what makes RAG the backbone of real, serious AI assistants, not just demos.



Chapter 3: The Core Pieces (and How They Fit Together)

We've got the big idea: *find the right information, then let the AI answer using it.* Now let's meet the pieces that actually make that happen. There are only five, and you already understand most of them.

Think of it as an assembly line for answers.

The five pieces

Here's each piece, in plain terms:

1. **Your documents.** Nelly's pile of notes, PDFs, articles. The raw knowledge. Nothing happens without this, it's *your* truth, the thing that makes the AI know *your* world.
2. **The memory (a vector store).** We can't hand the AI a thousand pages every time. So first we tidy the documents into a special kind of searchable memory. (We'll see exactly how in the next chapter: this is the part that feels like magic until it doesn't.)
3. **The retriever.** The part that, when a question comes in, dives into that memory and pulls out the few most relevant pieces. This is the "find" half of RAG.

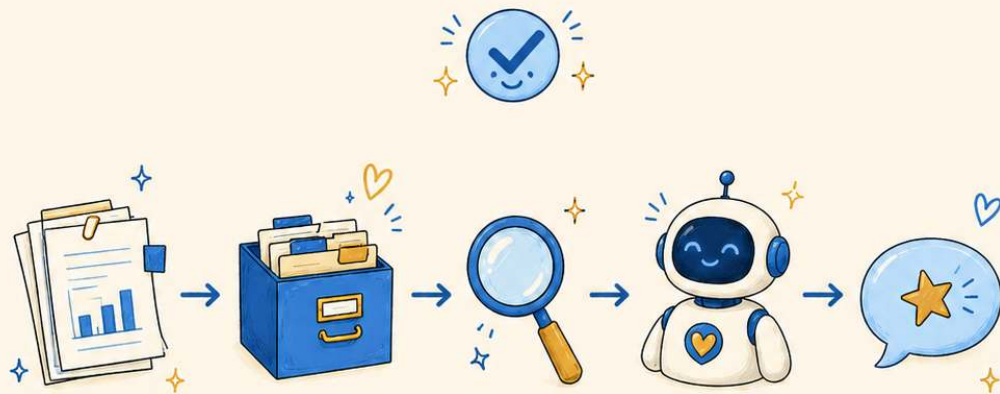


Putting it in a line

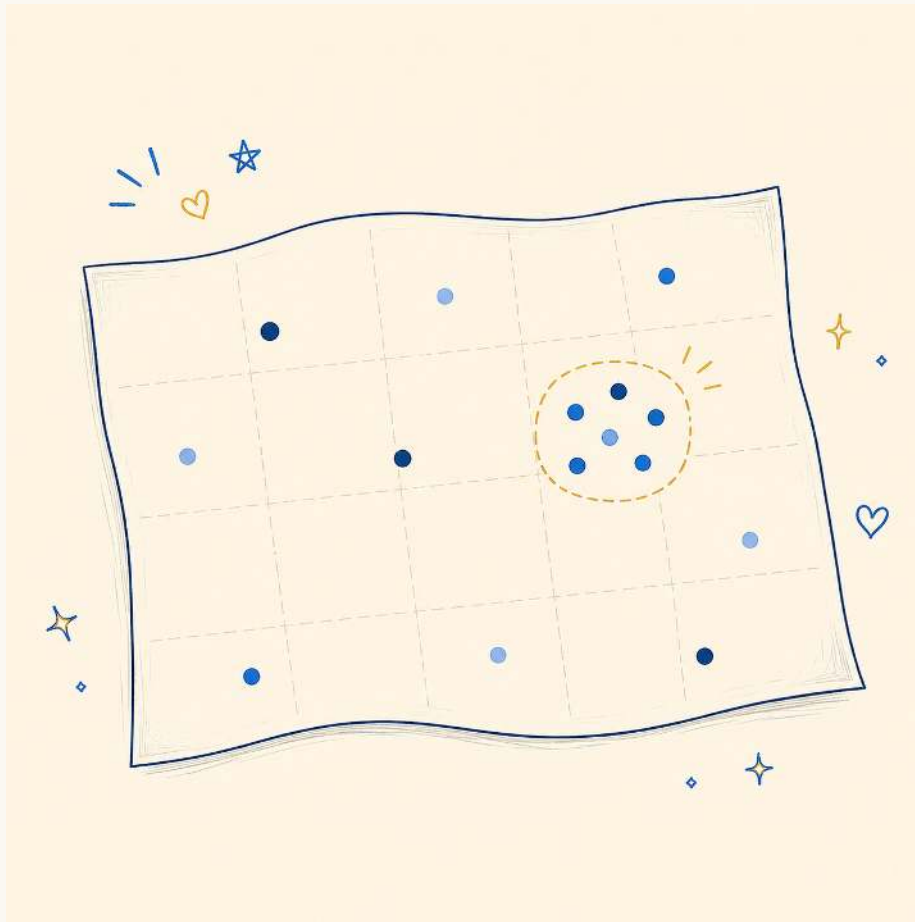
4. **The language model.** The AI that actually writes the answer, the “generate” half. But now, instead of guessing, it’s handed the retrieved pieces to answer from.
5. **The answer.** Grounded, specific, and traceable back to Nelly’s real documents. No more inventing.

Now read the whole line as one sentence:

Your documents become a searchable memory; a retriever finds the right pieces; the language model uses them to write a grounded answer.



That's the entire architecture. Every RAG system, from Nelly's little assistant to the giant ones running real companies, is just these five pieces done well. For the rest of the book, we'll zoom into the two that feel mysterious, the *memory* and the *retriever*, and then we'll build the whole thing.



Chapter 4: How Retrieval Works (Embeddings, Made Simple)

This is the chapter most books make confusing. We're going to make it click.

Here's the question we have to answer: when Nelly asks something, how does the computer *find* the right notes? It can't read like we do. It doesn't understand words the way we understand them.

So here's the trick computers use: they turn meaning into **location**.

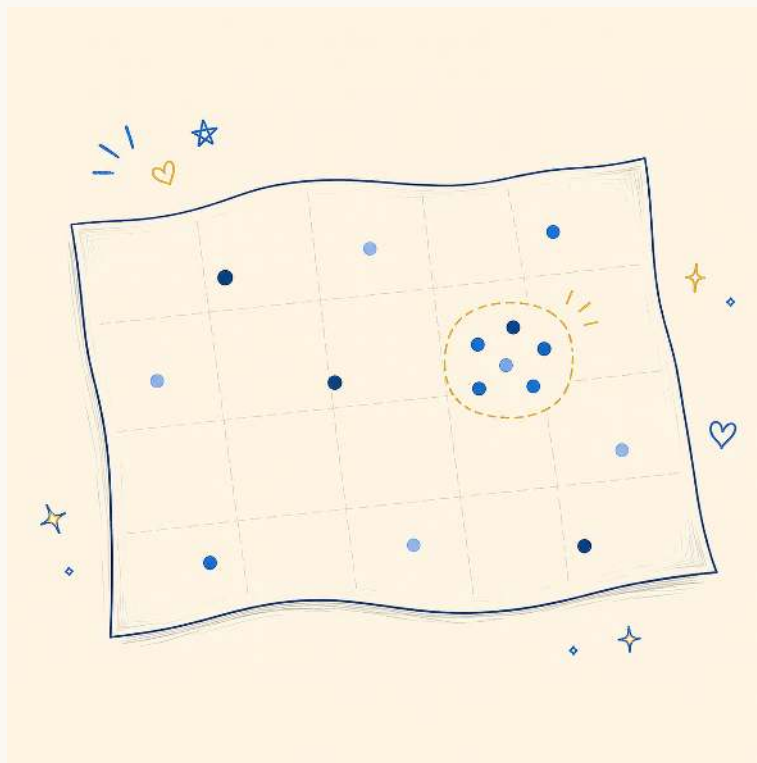
Meaning as a map

Imagine a giant map. On this map, every piece of text gets a spot. And the rule is simple:

Things that mean similar things sit close together.

Things that mean different things sit far apart.

So “cat” and “kitten” land right next to each other. “Cat” and “airplane” land far apart. Not because of the letters, but because of the *meaning*.



These map-coordinates have a name: **embeddings**. That's the whole scary word. An embedding is just a way of placing a piece of text on the meaning-map so similar things are neighbors.

Finding the nearest neighbors

Now the magic falls right out of it.

When Nelly asks a question, we place *her question* on the same map. Then we just look around its neighborhood and grab the notes sitting closest to it.

Those nearby notes are, by definition, the ones most *similar in meaning* to her question, which is exactly what she's looking for.



That's it. That's "vector search." A fancy name for a friendly idea: *put the question on the meaning-map, and grab its nearest neighbors.*

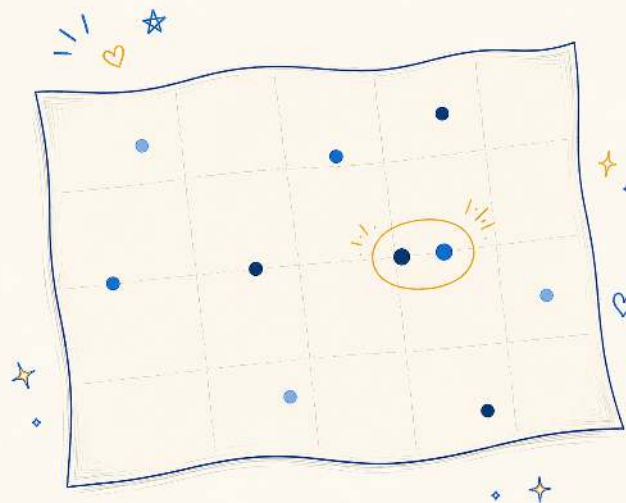
Why this beats Ctrl+F

Let's make sure it really landed, because everything else stands on this.

Old way (Ctrl+F): matches *exact words*. If Nelly searches "car" but her notes say "automobile," she finds nothing. The letters don't match.



New way (embeddings): matches *meaning*. "Car" and "automobile" sit right next to each other on the map, so it finds the note even though the words are different.



That's why RAG can find the right thing even when you don't use the exact words, it searches by *meaning*, not spelling. This single idea is the engine under the whole system. Nice work, you just understood the part most people never quite get.



Chapter 5: Letting the AI Answer (Grounding)

We can now *find* the right notes. But finding isn't answering. Nelly doesn't want three paragraphs dumped on her, she wants a clear answer. So how do we get from "here are the relevant notes" to "here's your answer"?

We hand the notes to the language model, and we ask it nicely.

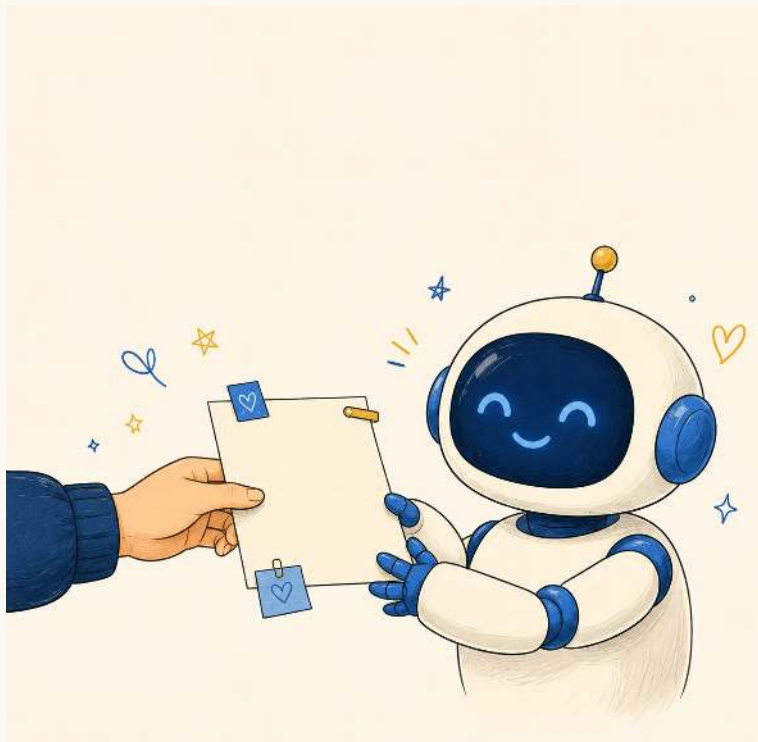
The one instruction that changes everything

Here's the actual move, and it's simpler than you'd think. Instead of asking the AI:

"What's the answer to Nelly's question?" (where it guesses from memory)

...we ask it:

"Here are the relevant notes. Using only these, answer Nelly's question."



That tiny change is everything. We're not relying on what the AI *happens* to know. We're handing it the truth and asking it to read from there. This is called **grounding**, the answer is *grounded* in real, retrieved information.

The whole machine, running

And now, watch the whole machine run, start to finish, with Nelly's question:



1. Nelly asks her question.
2. We place it on the meaning-map and grab the nearest notes.
3. We hand those notes to the AI with one instruction: answer using only these.
4. The AI writes a clear answer, built from her real documents.

No guessing. No inventing. Just her own knowledge, found and explained back to her.

That's a complete RAG system, in principle. In the next chapter, we stop describing it and actually build it.





Chapter 6: Build Your First RAG Assistant

Time to build. We're going to make a tiny, real RAG assistant for Nelly's notes.

A promise first: if you're not a coder, you can read straight past the code and still follow every step, the words explain what each piece does. And if you *do* want to build it, everything here actually runs.

We'll keep it to the smallest real version: a handful of notes, turned into a searchable memory, that we can ask questions of.

Step 1: Give it some notes

First, Nelly's documents. (Yours would be your own notes; we'll use a few short ones to keep it clear.)



```
notes = [  
    "The project deadline is March 3rd.",  
    "Water damage is not covered by the warranty.",  
    "To reset the device, hold the power button for 10 seconds.",  
]
```

That's our knowledge. In real life this would be Nelly's PDFs, loaded and split into bite-sized chunks, but the idea is identical: text, ready to be searched.

Steps 2 & 3: Build the memory, find the note

Step 2: Turn the notes into the meaning-map (embeddings). Remember the map where similar things sit close together? We create it like this:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
note_embeddings = model.encode(notes)
```



That's the "memory" piece from Chapter 3, done. Each note now has a spot on the map.

Step 3: Ask a question and find the nearest notes.

```
question = "How do I restart my device?"
question_embedding = model.encode([question])

# find the note closest in meaning
from sentence_transformers import util
hits = util.semantic_search(question_embedding, note_embeddings, top_k=1)
best_note = notes[hits[0][0]["corpus_id"]]
```

Notice: Nelly asked "restart," but the note says "reset." Different words, *same meaning*, so the meaning-map finds it anyway. Exactly the magic from Chapter 4.

Step 4: Let the AI answer

```
prompt = f"""Using only this note, answer the question.  
Note: {best_note}  
Question: {question}"""  
  
# send `prompt` to your language model of choice...
```



And that's it. That's a working RAG assistant. Notes → meaning-map → find the closest → hand it to the AI → grounded answer.

It's small, but every giant RAG system in the world is *this*, just with more notes, bigger models, and a proper memory store. You now understand and have built the real thing.

*You built it. That thing that
sounded impossible a few chapters
ago? Done.*



Chapter 7: When It Goes Wrong (and How to Fix It)

Here's something most tutorials skip, and it's the most useful chapter in the book: RAG doesn't always work perfectly the first time. Knowing *why* it breaks is what separates someone who *built a demo* from someone who *understands the system*.

Three things tend to go wrong. Let's meet them, and their fixes.

Problems 1 & 2

Problem 1: It retrieves the wrong notes.

If the “find” step grabs unrelated notes, the answer will be off, the AI can only work with what it's handed.

Fix: better chunking (split documents into clean, sensible pieces) and grabbing a few notes instead of just one, so the right one is more likely included.



Problem 2: It still makes things up.

Even grounded, an AI can sometimes wander off and add invented details.

Fix: a firmer instruction: “If the answer isn’t in the notes, say you don’t know.”

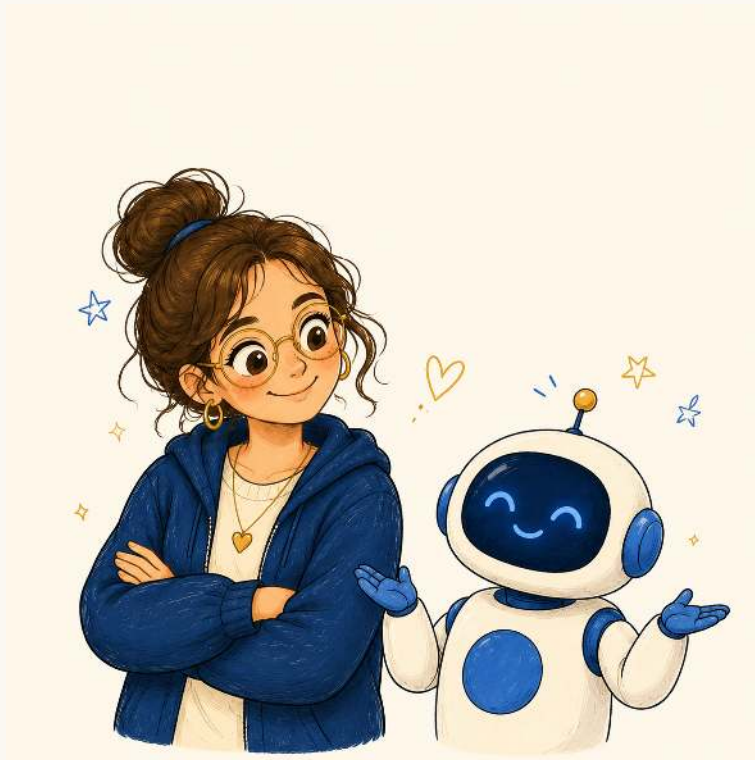
Permission to say “I don’t know” is what stops the inventing.

Problem 3, and the real lesson

Problem 3: The notes don't contain the answer at all.

Sometimes Nelly asks something her documents simply don't cover.

Fix: the system should *admit that*, not bluff. A good RAG assistant says "I couldn't find that in your notes" rather than guessing. Honesty is a feature.



Notice the theme: a *trustworthy* assistant is one that knows the limits of what it was given. The goal was never an AI that always has an answer. It's an AI whose answers you can *believe*, because they come from real notes, and it tells you honestly when they don't.

That's the difference between a party trick and a tool people rely on.



Chapter 8: Is It Any Good? (Evaluating Your Assistant)

You built it. It works. But “it works” is a feeling. How do you actually *know* it’s good? You check three simple things.

1. Did it find the right notes? (Retrieval)

Ask a question you know the answer to. Did it pull the correct note? If the *finding* is wrong, nothing downstream can be right, so this is the first thing to check.

The other two checks

2. Did the answer stick to the notes? (Faithfulness)

Read the answer against the retrieved note. Did it say only what's actually in there, or did it wander and invent? An answer can sound great and still be unfaithful; this catches that.

3. Did it actually answer the question? (Relevance)

A faithful answer that ignores what Nelly asked is still useless. Did it address her real question?



That's evaluation, no fancy math required to start: *found the right thing, stayed true to it, answered the question*. As your systems grow, there are deeper tools and metrics for each of these, but this honest three-question check is where every good evaluation begins. Get these three right and you have something genuinely trustworthy.



Chapter 9 - Where to Go Next

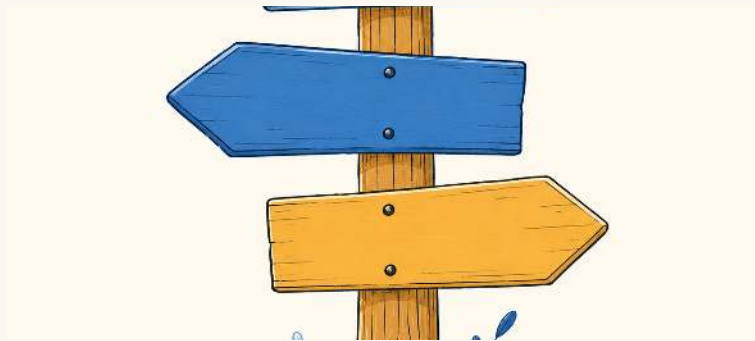
Look how far you've come.

A few chapters ago, RAG was three intimidating words. Now you know it's a simple, powerful idea, *find the right information, then let the AI answer using it*, and you've actually built one. Nelly's mountain of notes finally answers back, and so can yours.

The path from here

If you want to keep going, here's where the path leads:

- **Real documents:** load actual PDFs, split them into clean chunks, and point your assistant at them.
- **A real memory store:** swap our simple list for a proper vector database as your notes grow.
- **Better answers:** experiment with retrieving more notes, smarter chunking, and clearer instructions.
- **Your own assistant:** point it at *your* world, your studies, your work, your research, and let it answer from your real knowledge.



You don't need to learn it all at once. You already did the hard part, you understood the idea and built the thing. Everything from here is just more of what you already know.

That was the whole point. The scary stuff was never as out of reach as it looked.

Richel makes tech easy. And now, so do you.



A note from Richel

You just did something that sounded impossible a few pages ago. That's exactly what I love: taking the intimidating and making it doable.

If this clicked for you and you want to go further, learn more, build more, or work together, here's where to find me: [connect with me on LinkedIn](#).

No pressure. I'm just glad Nelly's notes (and yours) finally answer back.

Richel

Richel makes tech easy



Quick Reference

- **RAG = Find the right info, then let the AI answer using it.**
- The 5 pieces: documents → memory → retriever → language model → answer.
- Embeddings = meaning as location; similar things sit close.
- Vector search = put the question on the map, grab its nearest neighbors.
- Grounding = “answer using only these notes.”
- If it's not in the notes → say “I don't know.”
- Evaluate: found it? stayed true? answered it?

Keep learning



A few friendly pointers and links back to my work:

- Website: richelcode.github.io
- YouTube: [@richelattafuah23](https://www.youtube.com/channel/UC-richelattafuah23)
- Medium: [@richelattafuah](https://medium.com/@richelattafuah)
- GitHub: [RichelCode](https://github.com/RichelCode)

Richel makes tech easy



*You're about to build
something you thought was
out of reach. Let's make tech
easy, together.*



Richel Ohenewaa Attafuah

Richel makes tech easy